TECHNICAL ARCHITECTURE •
DOC 2 / 3

Backend Architecture & Data Model

The server side of iCare MC: the shared Supabase project, its PostgreSQL schema, storage, serverless functions and companion services.

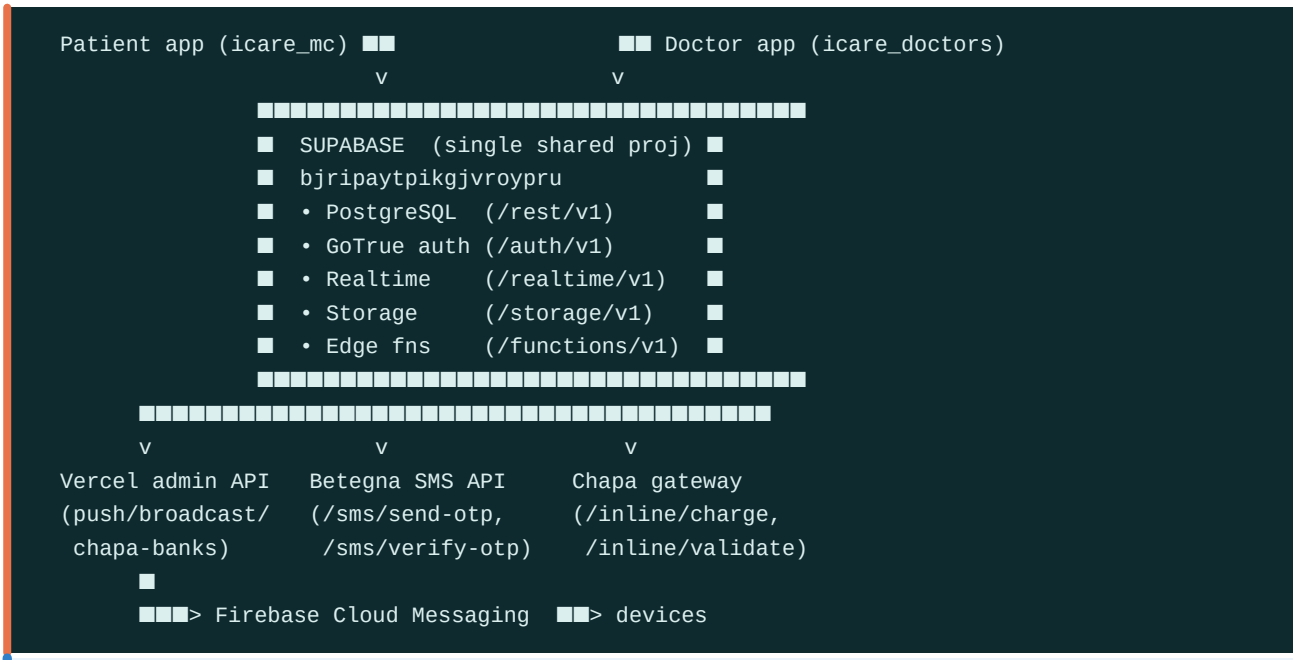
Subject	Shared backend for patient + doctor apps
Core	Supabase (PostgreSQL, GoTrue, Realtime, Storage, Edge)
Glue	Vercel serverless (admin/push, Chapa banks) + Betegna SMS
Payments	Chapa gateway with in-app wallet
Messaging	Firebase Cloud Messaging
Method	Inferred from client calls, table refs & column lists

Contents

1	Backend topology
2	Supabase as the system of record
3	The PostgreSQL data model
4	Row-Level Security & access model
5	Serverless / Edge Function responsibilities
6	Companion backends (SMS, push, Chapa banks)
7	Domain workflows end-to-end
8	Content, localisation & reference data
9	Operational concerns & hardening

1 Backend topology

iCare MC has no traditional application server that the apps own. The backend is a composition of managed platforms, with a single **Supabase project** at the centre acting as database, auth server, realtime broker, object store and function host. Two lightweight serverless surfaces on Vercel and an SMS gateway provide the glue that Supabase does not cover out of the box.



Single shared project, two audiences

Because both apps use the same Supabase project and anon key, the database schema itself must separate patient-visible and doctor-visible data using row policies and an `account type / role` attribute on the profile. This is the backbone of the whole design.

2 Supabase as the system of record

Supabase runs a managed PostgreSQL instance fronted by PostgREST. Client "repositories" (`supabase_repository.dart`, `doctor_repository.dart`, `chat_repository.dart`, `patient_wallet_repository.dart`, `review_repository.dart`) issue REST calls that the database answers directly, applying Row-Level Security per request.

Server-side responsibilities

- **Relational storage** of profiles, children, appointments, messages, services, wallet and content tables.
- **Identity** — GoTrue owns the users table, password hashing and JWT issuance/refresh.
- **Authorisation** — Postgres RLS policies decide which rows each authenticated user may read/write.
- **Realtime fan-out** — logical-replication stream powers chat and message subscriptions.
- **Object storage** — buckets for child photos, doctor profile images, service images and verification documents.
- **Business logic** — transactional operations are pushed into Postgres functions (RPC) and Edge Functions.

The client explicitly detects when the deployed schema is behind the app: *"Cloud database is missing tables or columns. Ask your admin to run the Supabase migrations, then try again."* This confirms a migration-driven schema managed outside the app.

3 The PostgreSQL data model

The tables below are reconstructed from table references, `select=` column lists, model `fromSupabaseRow / fromJson` factories and domain strings inside both binaries. Column lists shown are those directly observed; a real schema will contain additional bookkeeping columns (ids, timestamps).

3.1 Identity & people

Table	Represents	Key observed columns
profiles	Base account (patient or doctor)	id, account type/role, name, email, phone
doctor_profiles	Doctor-specific profile	first_name, last_name, specialty, hospital, phone, profile_photo_url
user_profile (model)	Patient profile view	name, email, phone, account type
hospitals	Hospital directory (doctor sign-up)	id, name, … (searchable field)

3.2 Child health domain (patient app)

Table	Represents	Key observed columns
children	A child record	id, user_id, local_id, name, birth data
child_growth_measurement	Weight/height entries	weight, height, symptoms, notes, blood_pressure_*, temperature
child_vaccine_record	Given/marked vaccines	user_id, child_local_id, vaccine_key, status
child_followup_visit	Scheduled follow-up visits	id, child_local_id, template_id, due_date, status, completed_at, checklist
child_followup_visit_templates	Visit definitions	code, label, sort_order, modules, vaccines
child_growth_periods	Age-band content	age_months, age_label, age_group, offset_days, growth_metrics
vaccine_dose_schedule	Dose calendar	code, display_name, dose_number, series_code, eligible_from/until_days

3.3 Pregnancy domain (patient app)

Table	Represents	Key observed columns
pregnancy_record	A tracked pregnancy	user_id, dates (LMP / due date)
pregnancy_week_log	Per-week journal	week, log data
pregnancy_weeks / _translations	Week content + i18n	week_number, image_url, trimester, title, subtitle, baby_development, mother_changes, recommendations, warning_signs, sections

3.4 Doctors, services & bookings

Table	Represents	Key observed columns
doctors	Doctor directory entry	id, profile fields, category
doctor_categories	Specialty categories	id, name, slug, sort_order
doctor_services	Bookable services	id, doctor_id, name, price
service_listing (model)	Rendered service card	service + doctor join
appointments	Booked appointments	appointment_date, time_slot, service_name, status, payment_status, total_amount
bookings	Booking transaction	booking + payment refs (Chapa tx_ref)
service_reviews	Ratings after a visit	rating, comment, service_id

3.5 Messaging, money & system

Table	Represents	Key observed columns
conversations	Chat thread (doctor ↔ patient)	id, participants
messages	Chat messages	conversation/thread id, sender, body, sent_at
patient_wallets	Patient wallet balance	user_id, balance
wallet_transactions	Wallet ledger (doctor payouts too)	amount, type, ref
bank_details	Doctor payout bank (Chapa)	bank, account, doctor_id
payment_settings	Chapa keys & fee config	public/secret key, fee percent
care_subscriptions	Care plan subscription	user_id, plan, status
daily_tips	Daily health tips (+translations)	tip_id, language_code, title, content
legal_documents	Terms / privacy content	type, body
app_settings (doctor)	Remote config / version policy	key, value
fcm_tokens	Device push tokens	user_id, fcm_token, platform

Offline-friendly IDs

Several child tables carry both a server id and a `local_id / child_local_id`. This dual-key pattern lets the app create records offline (guest/demo mode) and reconcile them with the server once the user signs in.

4 Row-Level Security & access model

Because the client only ever holds the public **anon** key plus a per-user JWT, all real authorisation must live in Postgres RLS. The expected policy shape, inferred from the data model:

- **Ownership** — patient tables (children, measurements, vaccines, wallet) are restricted to rows where `user_id = auth.uid()`.
- **Role split** — the `profiles` row carries an account-type/role; doctor-only tables (services, bank details, availability) are writable only by the owning doctor.
- **Shared conversations** — a message row is visible to the two participants of its conversation; appointments are visible to both the booking patient and the target doctor.
- **Public reference data** — content tables (tips, pregnancy weeks, categories, growth periods, legal docs) are world-readable under the anon key.

> **■ RLS is the only thing standing between users**
> With a shared anon key baked into a public APK, RLS is not optional. Any table lacking a correct policy is effectively public read/write. The shipped service-role key (see the API document) must be revoked because it bypasses RLS entirely.

5 Serverless / Edge Function responsibilities

Two serverless layers carry the logic that should not run on-device with the anon key:

5.1 Supabase Edge Functions (/functions/v1)

The `functions_client` package and the `/functions/v1` path are present. Edge Functions are the correct home for privileged, secret-bearing work: verifying a Chapa payment with the secret key, atomically finalising a booking + wallet credit, and any admin mutation. They run server-side with the service-role identity that must never be exposed to the client.

5.2 Vercel admin API (icaremc-admin)

A Next.js/Vercel deployment exposes push and broadcast endpoints plus a public Chapa-bank list. It is the piece that talks to FCM with a Firebase server key and, likely, to Chapa server-side.

Endpoint	Responsibility
`POST /api/bookings/push`	Push on appointment create / status change
`POST /api/chat/push`	Push the counterparty on a new chat message
`POST /api/broadcast`	Admin / topic broadcast announcements
`GET /api/public/chapa-banks`	Serve Chapa payout-bank options to doctors
`(webhook)`	‘chapa webhook update’, ‘chapa verify checked’, ‘chapa transfer sent’ log strings suggest payout/verify handling

6 Companion backends

6.1 Betegna AI SMS gateway

A dedicated OTP service issues and verifies SMS codes for phone verification and password recovery, decoupling SMS delivery (and its provider costs) from GoTrue. Endpoints `/sms/send-otp` and `/sms/verify-otp`.

6.2 Firebase Cloud Messaging

FCM is the delivery transport. Devices register a token, the app stores it in `fcm_tokens`, and the Vercel API asks FCM to deliver when events occur. The app renders alerts locally via `flutter_local_notifications`.

6.3 Chapa payment processor

Chapa handles the actual money movement (charge and, for doctors, payout transfers). The public key initiates checkout on-device; the secret key and verification/webhooks belong on the server. Doctor payout banks come from the Chapa-banks endpoint and are stored in `bank_details`.

7 Domain workflows end-to-end

7.1 Book & pay for an appointment

1. Patient opens a doctor → `doctors` + `doctor_services` read via PostgREST.
2. Patient picks a slot; pricing is computed (`booking_pricing_utils.dart`) incl. Chapa fee.
3. Chapa checkout runs on-device (`/inline/charge` → `/inline/validate`).
4. On success an `appointments` / `bookings` row is written with the Chapa `tx_ref` and payment_status.
5. The Vercel API pushes the doctor via FCM (`/api/bookings/push`).

7.2 Cancel & refund to wallet

1. Cancellation updates the appointment status.
2. A server function credits the paid amount (minus gateway fee) to `patient_wallets` / `wallet_transactions`.
3. User sees *"@amount was credited to your wallet"*.

7.3 Doctor–patient chat

1. Either side opens a `conversations` thread.
2. Messages are inserted into `messages` and streamed to the other side over Realtime.
3. A `/api/chat/push` call notifies the offline party via FCM.

7.4 Doctor onboarding & payout

1. Doctor signs up, completes personal details, uploads credentials to the `documents` / `credentials` buckets (`verify_documents`).
2. Doctor sets availability and services (`doctor_services`).
3. Earnings accrue; payout bank stored in `bank_details`; payouts settled through Chapa transfers with status tracked in `wallet_transactions`.

8 Content, localisation & reference data

A large part of the backend is **editorial content** served as data, so it can be updated without shipping an app release.

- **Localisation** — content tables carry sibling `*_translations` tables keyed by `language_code` (pregnancy weeks, growth periods, daily tips). Parsed by `content_translation_service.dart`. Oromo, Amharic and English strings are visible in the binary.
- **Pregnancy week content** — rich per-week guidance (baby development, mother changes, recommendations, warning signs).
- **Child growth periods** — age-banded milestones, vaccines, red-flags, nutrition and visit reminders.
- **WHO growth standards** — the LMS reference table ships *on-device* as `assets/who_lms/who_lms.json` (WHO Child Growth Standards, via nutriverse/zscorer) and is used by `who_zscore.dart` to compute z-scores locally.
- **Legal documents** and **version policy** are fetched from the DB so terms and force-update rules are server-controlled.

> **Hybrid data strategy**
> Static, universal science (WHO LMS tables, milestone definitions) is bundled in the app for offline use, while operational and editorial data (appointments, chat, tips, translations, legal text) lives in Supabase so it can change without an app update.

9 Operational concerns & hardening

Area	Current state (observed)	Recommendation
Secrets	service_role + access-token secret in patient .env	Revoke & rotate; keep secrets server-side only
Authorisation	Relies on Postgres RLS via anon key	Audit every table for a correct RLS policy
Payments	Chapa public key on client; verify server-side	Confirm all charge validation/webhooks run on server
Migrations	App detects missing tables/columns	Version schema; gate releases on migration state
Push	FCM via Vercel API + stored tokens	Rotate stale tokens; validate authorisation on push endpoints
Content	DB-driven with translations	Add caching / CDN for read-heavy public content
Observability	Client logs only (string evidence)	Centralise server logs/metrics for Vercel + Edge fns

Summary

The backend is a well-factored Backend-as-a-Service design: one shared Supabase PostgreSQL project provides data, identity, realtime, storage and functions; Vercel and Betegna add push, broadcast, Chapa-bank and SMS capabilities; Chapa and FCM handle money and notifications. The schema cleanly separates identity, child-health, pregnancy, doctor/booking, messaging, wallet and content domains. Correctness of Row-Level Security and removal of the shipped service-role key are the two decisive production-readiness items.

See companion documents: **API Architecture & Configuration** (1/3) and **User-Interface Architecture** (3/3).